

```

ThreadPoolTimer ^ PeriodicTimer = ThreadPoolTimer::CreatePeriodicTimer(
    ref new TimerElapsedHandler([this](ThreadPoolTimer^ source)
    {
        //
        // TODO: Work
        //

        //
        // Update the UI thread by using the UI core dispatcher.
        //
        Dispatcher->RunAsync(CoreDispatcherPriority::High,
            ref new DispatchedHandler([this]()
            {
                //
                // UI components can be accessed within this scope.
                //

                }));

    })),
    period,
    ref new TimerDestroyedHandler([&](ThreadPoolTimer ^ source)
    {
        //
        // TODO: Handle periodic timer cancellation.
        //

        Dispatcher->RunAsync(CoreDispatcherPriority::High,
            ref new DispatchedHandler([&]()
            {
                //
                // UI components can be accessed within this scope.
                //

                // Periodic timer cancelled.

                }));

    }));
}));

```

Cancel the timer

When necessary, call the [Cancel](#) method to stop the periodic work item from repeating. If the work item is running when the periodic timer is cancelled it is allowed to complete. The [TimerDestroyedHandler](#) (if provided) is called when all instances of the periodic work item have completed.

C#

```
PeriodicTimer.Cancel();
```

C++

```
PeriodicTimer->Cancel();
```

Remarks

For information about single-use timers, see [Use a timer to submit a work item](#).

Related topics

- [Submit a work item to the thread pool](#)
- [Best practices for using the thread pool](#)
- [Use a timer to submit a work item](#)

Submit a work item to the thread pool

Article • 10/27/2022

[Updated for UWP apps on Windows 10. For Windows 8.x articles, see the [archive](#)]

Important APIs

- [RunAsync](#)
- [IAsyncAction](#)

Learn how to do work in a separate thread by submitting a work item to the thread pool. Use this to maintain a responsive UI while still completing work that takes a noticeable amount of time, and use it to complete multiple tasks in parallel.

Create and submit the work item

Create a work item by calling [RunAsync](#). Supply a delegate to do the work (you can use a lambda, or a delegate function). Note that [RunAsync](#) returns an [IAsyncAction](#) object; store this object for use in the next step.

Three versions of [RunAsync](#) are available so that you can optionally specify the priority of the work item, and control whether it runs concurrently with other work items.

ⓘ Note

Use [CoreDispatcher.RunAsync](#) to access the UI thread and show progress from the work item.

The following example creates a work item and supplies a lambda to do the work:

C#

```
// The nth prime number to find.
const uint n = 9999;

// Receives the result.
ulong nthPrime = 0;

// Simulates work by searching for the nth prime number. Uses a
// naive algorithm and counts 2 as the first prime number.
IAsyncAction asyncAction = Windows.System.Threading.ThreadPool.RunAsync(
    (workItem) =>
    {
        uint progress = 0; // For progress reporting.
```

```

uint primes = 0;    // Number of primes found so far.
ulong i = 2;        // Number iterator.

if ((n >= 0) && (n <= 2))
{
    nthPrime = n;
    return;
}

while (primes < (n - 1))
{
    if (workItem.Status == AsyncStatus.Canceled)
    {
        break;
    }

    // Go to the next number.
    i++;

    // Check for prime.
    bool prime = true;
    for (uint j = 2; j < i; ++j)
    {
        if ((i % j) == 0)
        {
            prime = false;
            break;
        }
    };

    if (prime)
    {
        // Found another prime number.
        primes++;

        // Report progress at every 10 percent.
        uint temp = progress;
        progress = (uint)(10.0*primes/n);

        if (progress != temp)
        {
            String updateString;
            updateString = "Progress to " + n + "th prime: "
                + (10 * progress) + "%\n";

            // Update the UI thread with the CoreDispatcher.
            CoreApplication.MainView.CoreWindow.Dispatcher.RunAsync(
                CoreDispatcherPriority.High,
                new DispatchedHandler(() =>
                {
                    UpdateUI(updateString);
                }));
        }
    }
}

```

```

        // Return the nth prime number.
        nthPrime = i;
    });

    // A reference to the work item is cached so that we can trigger a
    // cancellation when the user presses the Cancel button.
    m_workItem = asyncAction;

```

C++/WinRT

```

// The nth prime number to find.
const unsigned int n{ 9999 };

// A shared pointer to the result.
// We use a shared pointer to keep the result alive until the
// work is done.
std::shared_ptr<unsigned long> nthPrime = std::make_shared<unsigned long>
(0);

// Simulates work by searching for the nth prime number. Uses a
// naive algorithm and counts 2 as the first prime number.

// A reference to the work item is cached so that we can trigger a
// cancellation when the user presses the Cancel button.
m_workItem = Windows::System::Threading::ThreadPool::RunAsync(
    [=, strongThis = get_strong()](Windows::Foundation::IAsyncAction const&
workItem)
{
    unsigned int progress = 0; // For progress reporting.
    unsigned int primes = 0;   // Number of primes found so far.
    unsigned long int i = 2;   // Number iterator.

    if ((n >= 0) && (n <= 2))
    {
        *nthPrime = n;
        return;
    }

    while (primes < (n - 1))
    {
        if (workItem.Status() == Windows::Foundation::AsyncStatus::Canceled)
        {
            break;
        }

        // Go to the next number.
        i++;

        // Check for prime.
        bool prime = true;
        for (unsigned int j = 2; j < i; ++j)
        {

```

```

        if ((i % j) == 0)
        {
            prime = false;
            break;
        }
    };

    if (prime)
    {
        // Found another prime number.
        primes++;

        // Report progress at every 10 percent.
        unsigned int temp = progress;
        progress = static_cast<unsigned int>(10.f*primes / n);

        if (progress != temp)
        {
            std::wstringstream updateStream;
            updateStream << L"Progress to " << n << L"th prime: " << (10
* progress) << std::endl;
            std::wstring updateString = updateStream.str();

            // Update the UI thread with the CoreDispatcher.

Windows::ApplicationModel::Core::CoreApplication::MainView().CoreWindow().Di
spatcher().RunAsync(
    Windows::UI::Core::CoreDispatcherPriority::High,
    Windows::UI::Core::DispatchedHandler([=]()
    {
        strongThis->UpdateUI(updateString);
    }));
    }
}
// Return the nth prime number.
*nthPrime = i;
});

```

C++/CX

```

// The nth prime number to find.
const unsigned int n = 9999;

// A shared pointer to the result.
// We use a shared pointer to keep the result alive until the
// work is done.
std::shared_ptr<unsigned long> nthPrime = std::make_shared<unsigned long>
(0);

// Simulates work by searching for the nth prime number. Uses a
// naive algorithm and counts 2 as the first prime number.
auto workItem = ref new Windows::System::Threading::WorkItemHandler(

```



```

    }
}
}
// Return the nth prime number.
*nthPrime = i;
});

auto asyncAction = ThreadPool::RunAsync(workItem);

// A reference to the work item is cached so that we can trigger a
// cancellation when the user presses the Cancel button.
m_workItem = asyncAction;

```

Following the call to [RunAsync](#), the work item is queued by the thread pool and runs when a thread becomes available. Thread pool work items run asynchronously and they can run in any order, so make sure your work items function independently.

Note that the work item checks the [IAsyncInfo.Status](#) property, and exits if the work item is cancelled.

Handle work item completion

Provide a completion handler by setting the [IAsyncAction.Completed](#) property of the work item. Supply a delegate (you can use a lambda or a delegate function) to handle work item completion. For example, use [CoreDispatcher.RunAsync](#) to access the UI thread and show the result.

The following example updates the UI with the result of the work item submitted in step 1:

C++/CX

```

asyncAction->Completed = ref new AsyncActionCompletedHandler(
    [this, n, nthPrime](IAsyncAction^ asyncInfo, AsyncStatus asyncStatus)
{
    if (asyncStatus == AsyncStatus::Canceled)
    {
        return;
    }

    String^ updateString;
    updateString = "\n" + "The " + n + "th prime number is "
        + (*nthPrime).ToString() + ".\n";

    // Update the UI thread with the CoreDispatcher.
   CoreApplication::MainView->CoreWindow->Dispatcher->RunAsync(
        CoreDispatcherPriority::High,
        ref new DispatchedHandler([this, updateString]()
        {

```



```

        UpdateUI(updateString);
    }));
});

```

C++/WinRT

```

m_workItem.Completed(
    [=, strongThis = get_strong()](Windows::Foundation::IAsyncAction const&
    asyncInfo, Windows::Foundation::AsyncStatus const& asyncStatus)
{
    if (asyncStatus == Windows::Foundation::AsyncStatus::Canceled)
    {
        return;
    }

    std::wstringstream updateStream;
    updateStream << std::endl << L"The " << n << L"th prime number is " <<
    *nthPrime << std::endl;
    std::wstring updateString = updateStream.str();

    // Update the UI thread with the CoreDispatcher.

    Windows::ApplicationModel::Core::CoreApplication::MainView().CoreWindow().Di
    spatcher().RunAsync(
        Windows::UI::Core::CoreDispatcherPriority::High,
        Windows::UI::Core::DispatchedHandler([=]()
        {
            strongThis->UpdateUI(updateString);
        }));
});

```

c#

```

asyncAction.Completed = new AsyncActionCompletedHandler(
    (IAsyncAction asyncInfo, AsyncStatus asyncStatus) =>
{
    if (asyncStatus == AsyncStatus.Canceled)
    {
        return;
    }

    String updateString;
    updateString = "\n" + "The " + n + "th prime number is "
        + nthPrime + ".\n";

    // Update the UI thread with the CoreDispatcher.
    CoreApplication.MainView.CoreWindow.Dispatcher.RunAsync(
        CoreDispatcherPriority.High,
        new DispatchedHandler(()=>
        {
            UpdateUI(updateString);
        }));
});

```

```
    }));  
});
```

Note that the completion handler checks whether the work item was cancelled before dispatching a UI update.

Summary and next steps

You can learn more by downloading the code from this quickstart in the [Creating a ThreadPool work item sample](#) written for Windows 8.1, and re-using the source code in a win_unap Windows 10 app.

Related topics

- [Submit a work item to the thread pool](#)
- [Best practices for using the thread pool](#)
- [Use a timer to submit a work item](#)

Use a timer to submit a work item

Article • 10/20/2022

Important APIs

- [Windows.UI.Core namespace](#)
- [Windows.System.Threading namespace](#)

Learn how to create a work item that runs after a timer elapses.

Create a single-shot timer

Use the [CreateTimer](#) method to create a timer for the work item. Supply a lambda that accomplishes the work, and use the *delay* parameter to specify how long the thread pool waits before it can assign the work item to an available thread. The delay is specified using a [TimeSpan](#) structure.

Note You can use [CoreDispatcher.RunAsync](#) to access the UI and show progress from the work item.

The following example creates a work item that runs in three minutes:

C#

```
TimeSpan delay = TimeSpan.FromMinutes(3);

ThreadPoolTimer DelayTimer = ThreadPoolTimer.CreateTimer(
    (source) =>
    {
        //
        // TODO: Work
        //

        //
        // Update the UI thread by using the UI core dispatcher.
        //
        Dispatcher.RunAsync(
            CoreDispatcherPriority.High,
            () =>
            {
                //
                // UI components can be accessed within this scope.
                //

            });
    });
```

```
}, delay);
```

C++

```
TimeSpan delay;
delay.Duration = 3 * 60 * 10000000; // 10,000,000 ticks per second

ThreadPoolTimer ^ DelayTimer = ThreadPoolTimer::CreateTimer(
    ref new TimerElapsedHandler([this](ThreadPoolTimer^ source)
    {
        //
        // TODO: Work
        //

        //
        // Update the UI thread by using the UI core dispatcher.
        //
        Dispatcher->RunAsync(CoreDispatcherPriority::High,
            ref new DispatchedHandler([this]()
            {
                //
                // UI components can be accessed within this scope.
                //

                ExampleUIUpdateMethod("Timer completed.");

            }));
    })), delay);
```

Provide a completion handler

If needed, handle cancellation and completion of the work item with a [TimerDestroyedHandler](#). Use the [CreateTimer](#) overload to supply an additional lambda. This runs when the timer is cancelled or when the work item completes.

The following example creates a timer that submits the work item, and calls a method when the work item finishes or the timer is cancelled:

C#

```
TimeSpan delay = TimeSpan.FromMinutes(3);

bool completed = false;

ThreadPoolTimer DelayTimer = ThreadPoolTimer.CreateTimer(
    (source) =>
    {
```

```

//
// TODO: Work
//

//
// Update the UI thread by using the UI core dispatcher.
//
Dispatcher.RunAsync(
    CoreDispatcherPriority.High,
    () =>
    {
        //
        // UI components can be accessed within this scope.
        //

    });

    completed = true;
},
delay,
(source) =>
{
    //
    // TODO: Handle work cancellation/completion.
    //

//
// Update the UI thread by using the UI core dispatcher.
//
Dispatcher.RunAsync(
    CoreDispatcherPriority.High,
    () =>
    {
        //
        // UI components can be accessed within this scope.
        //

        if (completed)
        {
            // Timer completed.
        }
        else
        {
            // Timer cancelled.
        }

    });

});

```

```

TimeSpan delay;
delay.Duration = 3 * 60 * 10000000; // 10,000,000 ticks per second

completed = false;

ThreadPoolTimer ^ DelayTimer = ThreadPoolTimer::CreateTimer(
    ref new TimerElapsedHandler([&](ThreadPoolTimer ^ source)
    {
        //
        // TODO: Work
        //

        //
        // Update the UI thread by using the UI core dispatcher.
        //
        Dispatcher->RunAsync(CoreDispatcherPriority::High,
            ref new DispatchedHandler([&]()
            {
                //
                // UI components can be accessed within this scope.
                //

                }));

        completed = true;

    })),
    delay,
    ref new TimerDestroyedHandler([&](ThreadPoolTimer ^ source)
    {
        //
        // TODO: Handle work cancellation/completion.
        //

        Dispatcher->RunAsync(CoreDispatcherPriority::High,
            ref new DispatchedHandler([&]()
            {
                //
                // Update the UI thread by using the UI core dispatcher.
                //

                if (completed)
                {
                    // Timer completed.
                }
                else
                {
                    // Timer cancelled.
                }

            }));

    }));
}));

```

Cancel the timer

If the timer is still counting down, but the work item is no longer needed, call [Cancel](#). The timer is cancelled and the work item won't be submitted to the thread pool.

C#

```
DelayTimer.Cancel();
```

C++

```
DelayTimer->Cancel();
```

Remarks

Universal Windows Platform (UWP) apps can't use **Thread.Sleep** because it can block the UI thread. You can use a [ThreadPoolTimer](#) to create a work item instead, and this will delay the task accomplished by the work item without blocking the UI thread.

See the [thread pool sample](#) [↗](#) for a complete code sample that demonstrates work items, timer work items, and periodic work items. The code sample was originally written for Windows 8.1 but the code can be re-used in Windows 10.

For information about repeating timers, see [Create a periodic work item](#).

Related topics

- [Submit a work item to the thread pool](#)
- [Best practices for using the thread pool](#)
- [Use a timer to submit a work item](#)

Using Windows Runtime objects in a multithreaded environment

Article • 10/20/2022

This article discusses the way the .NET Framework handles calls from C# and Visual Basic code to objects that are provided by the Windows Runtime or by Windows Runtime components.

In the .NET Framework, you can access any object from multiple threads by default, without special handling. All you need is a reference to the object. In the Windows Runtime, such objects are called *agile*. Most Windows Runtime classes are agile, but a few classes are not, and even agile classes may require special handling.

Wherever possible, the common language runtime (CLR) treats objects from other sources, such as the Windows Runtime, as if they were .NET Framework objects:

- If the object implements the [IAgileObject](#) interface, or has the [MarshalingBehaviorAttribute](#) attribute with [MarshalingType.Agile](#), the CLR treats it as agile.
- If CLR can marshal a call from the thread where it was made to the threading context of the target object, it does so transparently.
- If the object has the [MarshalingBehaviorAttribute](#) attribute with [MarshalingType.None](#), the class does not provide marshaling information. The CLR cannot marshal the call, so it throws an [InvalidCastException](#) exception with a message indicating that the object can be used only in the threading context where it was created.

The following sections describe the effects of this behavior on objects from various sources.

Objects from a Windows Runtime component that is written in C# or Visual Basic

All the types in the component that can be activated are agile by default.

📌 Note

Agility doesn't imply thread safety. In both the Windows Runtime and the .NET Framework, most classes are not thread safe because thread safety has a performance cost, and most objects are never accessed by multiple threads. It's more efficient to synchronize access to individual objects (or to use thread-safe classes) only as necessary.

When you author a Windows Runtime component, you can override the default. See the [ICustomQueryInterface](#) interface and the [IAgileObject](#) interface.

Objects from the Windows Runtime

Most classes in the Windows Runtime are agile, and the CLR treats them as agile. The documentation for these classes lists "MarshalingBehaviorAttribute(Agile)" among the class attributes. However, the members of some of these agile classes, such as XAML controls, throw exceptions if they aren't called on the UI thread. For example, the following code tries to use a background thread to set a property of the button that was clicked. The button's [Content](#) property throws an exception.

C#

```
private async void Button_Click_2(object sender, RoutedEventArgs e)
{
    Button b = (Button) sender;
    await Task.Run(() => {
        b.Content += ".";
    });
}
```

VB

```
Private Async Sub Button_Click_2(sender As Object, e As RoutedEventArgs)
    Dim b As Button = CType(sender, Button)
    Await Task.Run(Sub()
        b.Content &= "."
    End Sub)
End Sub
```

You can access the button safely by using its [Dispatcher](#) property, or the `Dispatcher` property of any object that exists in the context of the UI thread (such as the page the button is on). The following code uses the [CoreDispatcher](#) object's [RunAsync](#) method to dispatch the call on the UI thread.

C#

```
private async void Button_Click_2(object sender, RoutedEventArgs e)
{
    Button b = (Button) sender;
    await b.Dispatcher.RunAsync(
        Windows.UI.Core.CoreDispatcherPriority.Normal,
        () => {
            b.Content += ".";
        });
}
```

VB

```
Private Async Sub Button_Click_2(sender As Object, e As RoutedEventArgs)
    Dim b As Button = CType(sender, Button)
    Await b.Dispatcher.RunAsync(
        Windows.UI.Core.CoreDispatcherPriority.Normal,
        Sub()
            b.Content &= "."
        End Sub)
End Sub
```

ⓘ Note

The `Dispatcher` property does not throw an exception when it's called from another thread.

The lifetime of a Windows Runtime object that is created on the UI thread is bounded by the lifetime of the thread. Do not try to access objects on a UI thread after the window has closed.

If you create your own control by inheriting a XAML control, or by composing a set of XAML controls, your control is agile because it's a .NET Framework object. However, if it calls members of its base class or constituent classes, or if you call inherited members, those members will throw exceptions when they are called from any thread except the UI thread.

Classes that can't be marshaled

Windows Runtime classes that do not provide marshaling information have the [MarshalingBehaviorAttribute](#) attribute with [MarshalingType.None](#). The documentation for such a class lists "MarshalingBehaviorAttribute(None)" among its attributes.

The following code creates a [CameraCaptureUI](#) object on the UI thread, and then tries to set a property of the object from a thread pool thread. The CLR is unable to marshal the

call, and throws a [System.InvalidCastException](#) exception with a message indicating that the object can be used only in the threading context where it was created.

C#

```
Windows.Media.Capture.CameraCaptureUI ccui;

private async void Button_Click_1(object sender, RoutedEventArgs e)
{
    ccui = new Windows.Media.Capture.CameraCaptureUI();

    await Task.Run(() => {
        ccui.PhotoSettings.AllowCropping = true;
    });
}
```

VB

```
Private ccui As Windows.Media.Capture.CameraCaptureUI

Private Async Sub Button_Click_1(sender As Object, e As RoutedEventArgs)
    ccui = New Windows.Media.Capture.CameraCaptureUI()

    Await Task.Run(Sub()
        ccui.PhotoSettings.AllowCropping = True
    End Sub)
End Sub
```

The documentation for [CameraCaptureUI](#) also lists "ThreadingAttribute(STA)" among the class's attributes, because it must be created in a single-threaded context such as the UI thread.

If you want to access the [CameraCaptureUI](#) object from another thread, you can cache the [CoreDispatcher](#) object for the UI thread and use it later to dispatch the call on that thread. Or you can obtain the dispatcher from a XAML object such as the page, as shown in the following code.

C#

```
Windows.Media.Capture.CameraCaptureUI ccui;

private async void Button_Click_3(object sender, RoutedEventArgs e)
{
    ccui = new Windows.Media.Capture.CameraCaptureUI();

    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
        () => {
            ccui.PhotoSettings.AllowCropping = true;
        });
}
```

```
});  
}
```

VB

```
Dim ccui As Windows.Media.Capture.CameraCaptureUI  
  
Private Async Sub Button_Click_3(sender As Object, e As RoutedEventArgs)  
  
    ccui = New Windows.Media.Capture.CameraCaptureUI()  
  
    Await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,  
        Sub()  
            ccui.PhotoSettings.AllowCropping = True  
        End Sub)  
  
End Sub
```

Objects from a Windows Runtime component that is written in C++

By default, classes in the component that can be activated are agile. However, C++ allows a significant amount of control over threading models and marshaling behavior. As described earlier in this article, the CLR recognizes agile classes, tries to marshal calls when classes are not agile, and throws a [System.InvalidCastException](#) exception when a class has no marshaling information.

For objects that run on the UI thread and throw exceptions when they are called from a thread other than the UI thread, you can use the UI thread's [CoreDispatcher](#) object to dispatch the call.

See Also

[C# Guide](#)

[Visual Basic Guide](#)

User interface and input

Article • 07/15/2024

This section provides information about creating user interfaces and handing input for Universal Windows Platform (UWP) apps.

 Expand table

Topic	Description
Title bar customization	Shows how to create a custom title bar for your app.
Use Mica with WinUI 2 for UWP	Shows how to apply Mica as the base layer of your UWP app using WinUI 2.

See also

- [Develop UWP apps](#)

Feedback

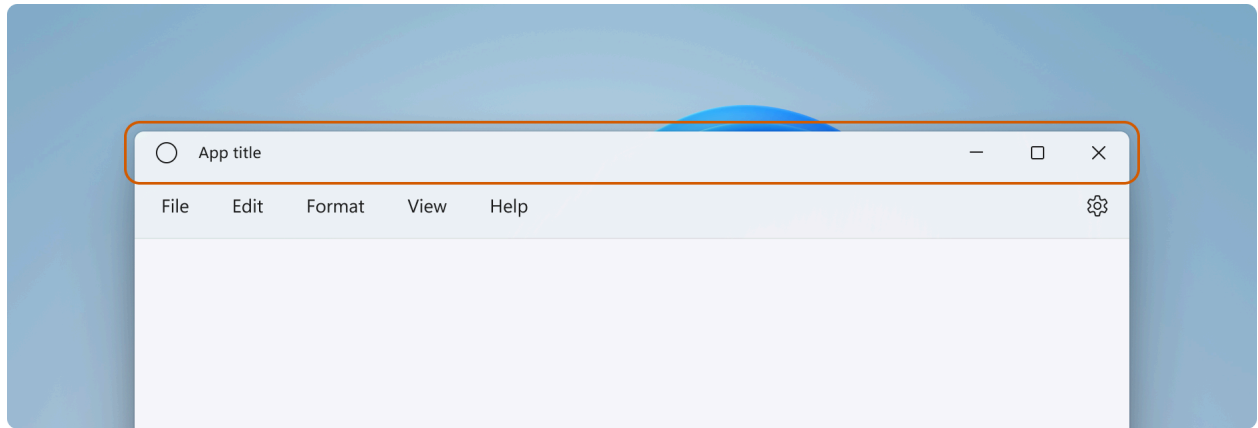
Was this page helpful?  

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Title bar customization

Article • 01/15/2024

Windows provides a default title bar for every window and lets you customize it to match the personality of your app. The default title bar comes with some standard components and core functionality such as dragging and resizing the window.



See the [Title bar](#) design article for guidance on customizing your app's title bar, acceptable title bar area content, and recommended UI patterns.

See the Windows 11 Fluent Design guidance for title bar

ⓘ Note

This article shows how to customize the title bar for apps that use UWP and WinUI 2. For apps that use the Windows App SDK and WinUI 3, see [Title bar customization](#) for the Windows App SDK.

If you are considering migrating your UWP app to Windows App SDK, please view our windowing functionality migration guide. See [Windowing functionality migration](#) for more information.

- ✓ **Applies to:** UWP/WinUI 2
- ✓ **Important APIs:** [ApplicationView.TitleBar](#) property, [ApplicationViewTitleBar](#) class, [CoreApplicationViewTitleBar](#) class

Title bar components

This list describes the components of the standard title bar.

- Title bar rectangle

- Title text
- System menu - accessed by clicking the app icon or right-clicking the title bar
- Caption controls
 - Minimize button
 - Maximize/Restore button
 - Close button

In UWP applications, you can customize the title bar by using members of the [ApplicationView](#) and [CoreApplicationView](#) classes. There are multiple APIs to progressively modify the appearance of your title bar based on the level of customization needed.

ⓘ Note

The [Windows.UI.WindowManagement.AppWindow](#) class, used for secondary windows in UWP apps, does not support title bar customization. To customize the title bar of a UWP app that uses secondary windows, use [ApplicationView](#) as described in [Show multiple views with ApplicationView](#).

How much to customize the title bar

There are two levels of customization that you can apply to the title bar: apply minor modifications to the default title bar, or extend your app canvas into the title bar area and provide completely custom content.

Simple

For simple customization, such as changing the title bar color, you can set properties on your app window's title bar object to specify the colors you want to use for title bar elements. In this case, the system retains responsibility for all other aspects of the title bar, such as drawing the app title and defining drag areas.

Full

Your other option is to hide the default title bar and replace it with your own custom content. For example, you can place text, a search box, or custom menus in the title bar area. You will also need to use this option to extend a [material](#) backdrop, like [Mica](#), into the title bar area.

When you opt for full customization, you are responsible for putting content in the title bar area, and you can define your own drag region. The caption controls (system Close, Minimize, and Maximize buttons) are still available and handled by the system, but elements like the app title are not. You will need to create those elements yourself as needed by your app.

Simple customization

If you want only to customize the title bar colors or icon, you can set properties on the title bar object for your app window.

Title

By default, the title bar shows the app's display name as the window title. The display name is set in the `Package.appxmanifest` file.

To add custom text to the title, set the `ApplicationView.Title` property to a text value, as shown here.

C#

```
public MainPage()
{
    this.InitializeComponent();

    ApplicationView.GetForCurrentView().Title = "Custom text";
}
```

Your text is prepended to the window title, which will be displayed as "*custom text - app display name*". To show a custom title without the app display name, you have to replace the default title bar as shown in the [Full customization](#) section.

Colors

This example shows how to get an instance of `ApplicationViewTitleBar` and set its color properties.

This code can be placed in your app's `OnLaunched` method (`App.xaml.cs`), after the call to `Window.Activate`, or in your app's first page.

C#


```
// using Windows.UI;
// using Windows.UI.ViewManagement;

var titleBar = ApplicationView.GetForCurrentView().TitleBar;

// Set active window colors
titleBar.ForegroundColor = Colors.White;
titleBar.BackgroundColor = Colors.Green;
titleBar.ButtonForegroundColor = Colors.White;
titleBar.ButtonBackgroundColor = Colors.SeaGreen;
titleBar.ButtonHoverForegroundColor = Colors.White;
titleBar.ButtonHoverBackgroundColor = Colors.DarkSeaGreen;
titleBar.ButtonPressedForegroundColor = Colors.Gray;
titleBar.ButtonPressedBackgroundColor = Colors.LightGreen;

// Set inactive window colors
titleBar.InactiveForegroundColor = Colors.Gainsboro;
titleBar.InactiveBackgroundColor = Colors.SeaGreen;
titleBar.ButtonInactiveForegroundColor = Colors.Gainsboro;
titleBar.ButtonInactiveBackgroundColor = Colors.SeaGreen;
```

There are a few things to be aware of when setting title bar colors:

- The button background color is not applied to the close button *hover* and *pressed* states. The close button always uses the system-defined color for those states.
- Setting a color property to `null` resets it to the default system color.
- You can't set transparent colors. The color's alpha channel is ignored.

Windows gives a user the option to apply their selected [accent color](#) to the title bar. If you set any title bar color, we recommend that you explicitly set all the colors. This ensures that there are no unintended color combinations that occur because of user defined color settings.

Full customization

When you opt-in to full title bar customization, your app's client area is extended to cover the entire window, including the title bar area. You are responsible for drawing and input-handling for the entire window except the caption buttons, which are still provided by the window.

To hide the default title bar and extend your content into the title bar area, set the [ExtendViewIntoTitleBar](#) property to `true`. You can set this property in your app's `OnLaunched` method (*App.xaml.cs*), or in your app's first page.



Tip

See the [Full customization example](#) section to see all the code at once.

This example shows how to get the `CoreApplicationViewTitleBar` and set the `ExtendViewIntoTitleBar` property to `true`.

C#

```
using Windows.ApplicationModel.Core;

public MainPage()
{
    this.InitializeComponent();

    // Hide default title bar.
    var coreTitleBar = CoreApplication.GetCurrentView().TitleBar;
    coreTitleBar.ExtendViewIntoTitleBar = true;
}
```

💡 Tip

This setting persists when your app is closed and restarted. In Visual Studio, if you set `ExtendViewIntoTitleBar` to `true`, and then want to revert to the default, you should explicitly set it to `false` and run your app to overwrite the persisted setting.

Title bar content and drag regions

When your app is extended into the title bar area, you're responsible for defining and managing the UI for the title bar. This typically includes, at a minimum, specifying title text and the drag region. The drag region of the title bar defines where the user can click and drag to move the window around. It's also where the user can right-click to show the system menu.

To learn more about acceptable title bar content and recommended UI patterns, see [Title bar design](#).

You specify the drag region by calling the `Window.SetTitleBar` method and passing in a `UIElement` that defines the drag region. (The `UIElement` is typically a panel that contains other elements.) The `ExtendViewIntoTitleBar` property must be set to `true` in order for the call to `SetTitleBar` to have any effect.

Here's how to set a `Grid` of content as the draggable title bar region. This code goes in the XAML and code-behind for your app's first page.

XAML

```
<Grid x:Name="AppTitleBar" Background="Transparent">
    <!-- Width of the padding columns is set in LayoutMetricsChanged
    handler. -->
    <!-- Using padding columns instead of Margin ensures that the background
    paints the area under the caption control buttons (for transparent
    buttons). -->
    <Grid.ColumnDefinitions>
        <ColumnDefinition x:Name="LeftPaddingColumn" Width="0"/>
        <ColumnDefinition/>
        <ColumnDefinition x:Name="RightPaddingColumn" Width="0"/>
    </Grid.ColumnDefinitions>
    <Image Source="Assets/WindowIcon.png"
        Grid.Column="1"
        HorizontalAlignment="Left"
        Width="16" Height="16"
        Margin="8,0,0,0"/>
    <TextBlock x:Name="AppTitleTextBlock"
        Text="App title"
        Style="{StaticResource CaptionTextBlockStyle}"
        Grid.Column="1"
        VerticalAlignment="Center"
        Margin="28,0,0,0"/>
</Grid>
```

C#

```
public MainPage()
{
    this.InitializeComponent();

    var coreTitleBar = CoreApplication.GetCurrentView().TitleBar;
    coreTitleBar.ExtendViewIntoTitleBar = true;

    // Set XAML element as a drag region.
    Window.Current.SetTitleBar(AppTitleBar);
}
```

By default, the system title bar shows the app's display name as the window title. The display name is set in the Package.appxmanifest file. You can get this value and use it in your custom title bar like this.

C#

```
AppTitleTextBlock.Text = AppInfo.Current.DisplayName;
```

❗ Important

The drag region you specify needs to be hit testable. By default, some UI elements, such as `Grid`, do not participate in hit testing when they don't have a background set. This means, for some elements, you might need to set a transparent background brush. See the remarks on [VisualTreeHelper.FindElementsInHostCoordinates](#) for more info.

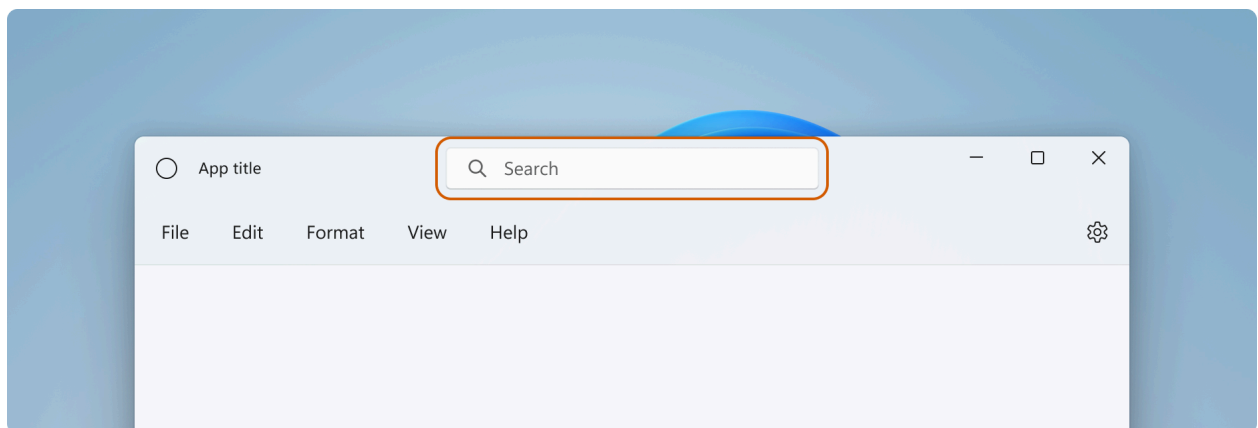
For example, if you define a `Grid` as your drag region, set `Background="Transparent"` to make it draggable.

This `Grid` is not draggable (but visible elements within it are): `<Grid x:Name="AppTitleBar">.`

This `Grid` looks the same, but the whole `Grid` is draggable: `<Grid x:Name="AppTitleBar" Background="Transparent">.`

Interactive content

You can place interactive controls, like buttons, menus, or a search box, in the top part of the app so they appear to be in the title bar. However, there are a few rules you must follow to ensure that your interactive elements receive user input while still allowing users to move your window around.



- You must call `SetTitleBar` to define an area as the draggable title bar region. If you don't, the system sets the default drag region at the top of the page. The system will then handle all user input to this area, and prevent input from reaching your controls.
- Place your interactive controls over the top of the drag region defined by the call to `SetTitleBar` (with a higher z-order). Don't make your interactive controls children of the `UIElement` passed to `SetTitleBar`. After you pass an element to `SetTitleBar`, the system treats it like the system title bar and handles all pointer input to that element.

Here, the `AutoSuggestBox` element has a higher z-order than `AppTitleBar`, so it receives user input.

XAML

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="48"/>
        <RowDefinition/>
    </Grid.RowDefinitions>
    <Grid x:Name="AppTitleBar" Background="Transparent">
        <!-- Width of the padding columns is set in LayoutMetricsChanged
        handler. -->
        <!-- Using padding columns instead of Margin ensures that the
        background
            paints the area under the caption control buttons (for
        transparent buttons). -->
        <Grid.ColumnDefinitions>
            <ColumnDefinition x:Name="LeftPaddingColumn" Width="0"/>
            <ColumnDefinition/>
            <ColumnDefinition x:Name="RightPaddingColumn" Width="0"/>
        </Grid.ColumnDefinitions>
        <Image Source="Assets/WindowIcon.png"
            Grid.Column="1"
            HorizontalAlignment="Left"
            Width="16" Height="16"
            Margin="8,0,0,0"/>
        <TextBlock x:Name="AppTitleTextBlock"
            Text="App title"
            Style="{StaticResource CaptionTextBlockStyle}"
            Grid.Column="1"
            VerticalAlignment="Center"
            Margin="28,0,0,0"/>
    </Grid>

    <!-- This control has a higher z-order than AppTitleBar,
    so it receives user input. -->
    <AutoSuggestBox QueryIcon="Find"
        PlaceholderText="Search"
        HorizontalAlignment="Center"
        Width="260" Height="32"/>
</Grid>
```

System caption buttons

The system reserves the upper-left or upper-right corner of the app window for the system caption buttons (minimize, maximize/restore, close). The system retains control of the caption button area to guarantee that minimum functionality is provided for dragging, minimizing, maximizing, and closing the window. The system draws the Close

button in the upper-right for left-to-right languages and the upper-left for right-to-left languages.

You can draw content underneath the caption control area, such as your app background, but you should not put any UI that you expect the user to be able to interact with. It does not receive any input because input for the caption controls is handled by the system.

These lines from the previous example show the padding columns in the XAML that defines the title bar. Using padding columns instead of margins ensures that the background paints the area under the caption control buttons (for transparent buttons). Using both right and left padding columns ensures that your title bar behaves correctly in both right-to-left and left-to-right layouts.

XAML

```
<Grid.ColumnDefinitions>
    <ColumnDefinition x:Name="LeftPaddingColumn" Width="0"/>
    <ColumnDefinition/>
    <ColumnDefinition x:Name="RightPaddingColumn" Width="0"/>
</Grid.ColumnDefinitions>
```

The dimensions and position of the caption control area is communicated by the [CoreApplicationViewTitleBar](#) class so that you can account for it in the layout of your title bar UI. The width of the reserved region on each side is given by the [SystemOverlayLeftInset](#) or [SystemOverlayRightInset](#) properties, and its height is given by the [Height](#) property.

You can handle the [LayoutMetricsChanged](#) event to respond to changes in the size of the caption buttons. For example, this can happen if the app layout changes from left-to-right to right-to-left. Handle this event to verify and update the positioning of UI elements that depend on the title bar's size.

This example shows how to adjust the layout of your title bar to account for changes in the title bar metrics. `AppTitleBar`, `LeftPaddingColumn`, and `RightPaddingColumn` are declared in the XAML shown previously.

C#

```
private void CoreTitleBar_LayoutMetricsChanged(CoreApplicationViewTitleBar
sender, object args)
{
    // Get the size of the caption controls and set padding.
    LeftPaddingColumn.Width = new
GridLength(coreTitleBar.SystemOverlayLeftInset);
    RightPaddingColumn.Width = new
```

```
GridLength(coreTitleBar.SystemOverlayRightInset);  
}
```

Color and transparency in caption buttons

When you extend your app content into the title bar area, you can make the background of the caption buttons transparent to let your app background show through. You typically set the background to [Colors.Transparent](#) for full transparency. For partial transparency, set the alpha channel for the [Color](#) you set the property to.

These title bar properties can be transparent:

- [ButtonBackgroundColor](#)
- [ButtonHoverBackgroundColor](#)
- [ButtonPressedBackgroundColor](#)
- [ButtonInactiveBackgroundColor](#)

All other color properties will continue to ignore the alpha channel. If

`ExtendViewIntoTitleBar` is set to `false`, the alpha channel is always ignored for all `ApplicationViewTitleBar` color properties.

The button background color is not applied to the Close button *hover* and *pressed* states. The close button always uses the system-defined color for those states.

Tip

[Mica](#) is a delightful [material](#) that helps distinguish the window that's in focus. We recommend it as the background for long-lived windows in Windows 11. If you have applied Mica in the client area of your window, you can extend it into the title bar area and make your caption buttons transparent for the Mica to show through. See [Mica material](#) for more info.

Dim the title bar when the window is inactive

You should make it obvious when your window is active or inactive. At a minimum, you should change the color of the text, icons, and buttons in your title bar.

Handle the [CoreWindow.Activated](#) event to determine the activation state of the window, and update your title bar UI as needed.

```

public MainPage()
{
    ...
    Window.Current.CoreWindow.Activated += CoreWindow_Activated;
}

private void CoreWindow_Activated(CoreWindow sender,
WindowActivatedEventArgs args)
{
    UISettings settings = new UISettings();
    if (args.WindowActivationState == CoreWindowActivationState.Deactivated)
    {
        AppTitleTextBlock.Foreground =
            new
SolidColorBrush(settings.UIElementColor(UIElementType.GrayText));
    }
    else
    {
        AppTitleTextBlock.Foreground =
            new
SolidColorBrush(settings.UIElementColor(UIElementType.WindowText));
    }
}

```

Reset the title bar

You can call [SetTitleBar](#) to switch to a new title bar element while your app is running. You can also pass `null` as the parameter to `SetTitleBar` and set [ExtendViewIntoTitleBar](#) to `false` to revert to the default system title bar.

Show and hide the title bar

If you add support for *full screen* or *compact overlay* modes to your app, you might need to make changes to your title bar when your app switches between these modes.

When your app runs in *full screen* or *tablet mode (Windows 10 only)*, the system hides the title bar and caption control buttons. However, the user can invoke the title bar to have it shown as an overlay on top of the app's UI.

You can handle the [CoreApplicationViewTitleBar.IsVisibleChanged](#) event to be notified when the title bar is hidden or invoked, and show or hide your custom title bar content as needed.

This example shows how to handle the `IsVisibleChanged` event to show and hide the `AppTitleBar` element from previous examples.

C#

```
public MainPage()
{
    this.InitializeComponent();

    var coreTitleBar = CoreApplication.GetCurrentView().TitleBar;

    // Register a handler for when the title bar visibility changes.
    // For example, when the title bar is invoked in full screen mode.
    coreTitleBar.IsVisibleChanged += CoreTitleBar_IsVisibleChanged;
}

private void CoreTitleBar_IsVisibleChanged(CoreApplicationViewTitleBar
sender, object args)
{
    if (sender.IsVisible)
    {
        AppTitleBar.Visibility = Visibility.Visible;
    }
    else
    {
        AppTitleBar.Visibility = Visibility.Collapsed;
    }
}
```

ⓘ Note

Full screen mode can be entered only if supported by your app. See [ApplicationView.IsFullScreenMode](#) for more info. ***Tablet mode (Windows 10 only)*** is a user option in Windows 10 on supported hardware, so a user can choose to run any app in tablet mode.

Do's and don'ts

- Do make it obvious when your window is active or inactive. At a minimum, change the color of the text, icons, and buttons in your title bar.
- Do define a drag region along the top edge of the app canvas. Matching the placement of system title bars makes it easier for users to find.
- Do define a drag region that matches the visual title bar (if any) on the app's canvas.

Full customization example

This examples shows all the code described in the Full customization section.

```

<Page
    x:Class="WinUI2_ExtendedTitleBar.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:WinUI2_ExtendedTitleBar"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d"
    xmlns:muxc="using:Microsoft.UI.Xaml.Controls"
    muxc:BackdropMaterial.ApplyToRootOrPageBackground="True">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="48"/>
            <RowDefinition/>
        </Grid.RowDefinitions>
        <Grid x:Name="AppTitleBar" Background="Transparent">
            <!-- Width of the padding columns is set in LayoutMetricsChanged
handler. -->
            <!-- Using padding columns instead of Margin ensures that the
background
                paints the area under the caption control buttons (for
transparent buttons). -->
            <Grid.ColumnDefinitions>
                <ColumnDefinition x:Name="LeftPaddingColumn" Width="0"/>
                <ColumnDefinition/>
                <ColumnDefinition x:Name="RightPaddingColumn" Width="0"/>
            </Grid.ColumnDefinitions>
            <Image Source="Assets/WindowIcon.png"
                Grid.Column="1"
                HorizontalAlignment="Left"
                Width="16" Height="16"
                Margin="8,0,0,0"/>
            <TextBlock x:Name="AppTitleTextBlock"
                Text="App title"
                Style="{StaticResource CaptionTextBlockStyle}"
                Grid.Column="1"
                VerticalAlignment="Center"
                Margin="28,0,0,0"/>
        </Grid>

        <!-- This control has a higher z-order than AppTitleBar,
            so it receives user input. -->
        <AutoSuggestBox QueryIcon="Find"
            PlaceholderText="Search"
            HorizontalAlignment="Center"
            Width="260" Height="32"/>

        <muxc:NavigationView Grid.Row="1"
            IsBackButtonVisible="Collapsed"
            IsSettingsVisible="False">
            <StackPanel>
                <TextBlock Text="Content"

```

```

                Style="{ThemeResource TitleTextBlockStyle}"
                Margin="12,0,0,0"/>
            </StackPanel>
        </muxc:NavigationView>
    </Grid>
</Page>

```

C#

```

public MainPage()
{
    this.InitializeComponent();

    // Hide default title bar.
    CoreApplicationViewTitleBar coreTitleBar =
        CoreApplication.GetCurrentView().TitleBar;
    coreTitleBar.ExtendViewIntoTitleBar = true;

    // Set caption buttons background to transparent.
    ApplicationViewTitleBar titleBar =
        ApplicationView.GetForCurrentView().TitleBar;
    titleBar.ButtonBackgroundColor = Colors.Transparent;

    // Set XAML element as a drag region.
    Window.Current.SetTitleBar(AppTitleBar);

    // Register a handler for when the size of the overlaid caption control
    changes.
    coreTitleBar.LayoutMetricsChanged += CoreTitleBar_LayoutMetricsChanged;

    // Register a handler for when the title bar visibility changes.
    // For example, when the title bar is invoked in full screen mode.
    coreTitleBar.IsVisibleChanged += CoreTitleBar_IsVisibleChanged;

    // Register a handler for when the window activation changes.
    Window.Current.CoreWindow.Activated += CoreWindow_Activated;
}

private void CoreTitleBar_LayoutMetricsChanged(CoreApplicationViewTitleBar
sender, object args)
{
    // Get the size of the caption controls and set padding.
    LeftPaddingColumn.Width = new
GridLength(coreTitleBar.SystemOverlayLeftInset);
    RightPaddingColumn.Width = new
GridLength(coreTitleBar.SystemOverlayRightInset);
}

private void CoreTitleBar_IsVisibleChanged(CoreApplicationViewTitleBar
sender, object args)
{
    if (sender.IsVisible)
    {

```

```

        AppTitleBar.Visibility = Visibility.Visible;
    }
    else
    {
        AppTitleBar.Visibility = Visibility.Collapsed;
    }
}

private void CoreWindow_Activated(CoreWindow sender,
WindowActivatedEventArgs args)
{
    UISettings settings = new UISettings();
    if (args.WindowActivationState ==
CoreWindowActivationState.Deactivated)
    {
        AppTitleTextBlock.Foreground =
            new
SolidColorBrush(settings.UIElementColor(UIElementType.GrayText));
    }
    else
    {
        AppTitleTextBlock.Foreground =
            new
SolidColorBrush(settings.UIElementColor(UIElementType.WindowText));
    }
}

```

Related articles

- [Title bar customization \(Windows App SDK\)](#)
- [Acrylic](#)
- [Mica](#)
- [Color](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Apply Mica with WinUI 2 for UWP

Article • 07/15/2024

This article describes how to apply Mica as the base layer of your UWP app using WinUI 2, prioritizing application and visibility in the title bar area. For more information about app layering with Mica, see [Mica material](#).

Mica is an opaque material that incorporates the user's theme and desktop wallpaper to create a highly personalized appearance. As the user moves the window across the screen, the Mica material dynamically adapts to create a rich visualization using the wallpaper underneath the application. In addition, the material helps users focus on the current task by falling back to a neutral color when the app is inactive.

Mica is available for UWP apps that use [WinUI 2](#) while running on Windows 11 version 22000 or later. On Windows 10, the background falls back to a solid color.

Prerequisites

- The WinUI 2 NuGet package (see [Getting started with WinUI 2](#))
- Familiarity with [Mica material](#).
- Familiarity with [Title bar customization](#)

Use Mica with WinUI 2 for UWP

Important APIs: [BackdropMaterial class](#)

To apply Mica in a UWP app, you use the [BackdropMaterial](#) class from WinUI 2. We recommend that you set the [BackdropMaterial.ApplyToRootOrPageBackground](#) attached property on a XAML element that is the root of your XAML content, as it will apply to the entire content region (such as a Window). If your app has a [Frame](#) that navigates multiple pages, you should set this property on the Frame. Otherwise, you should set this property on the main [Page](#) of your app.

XAML

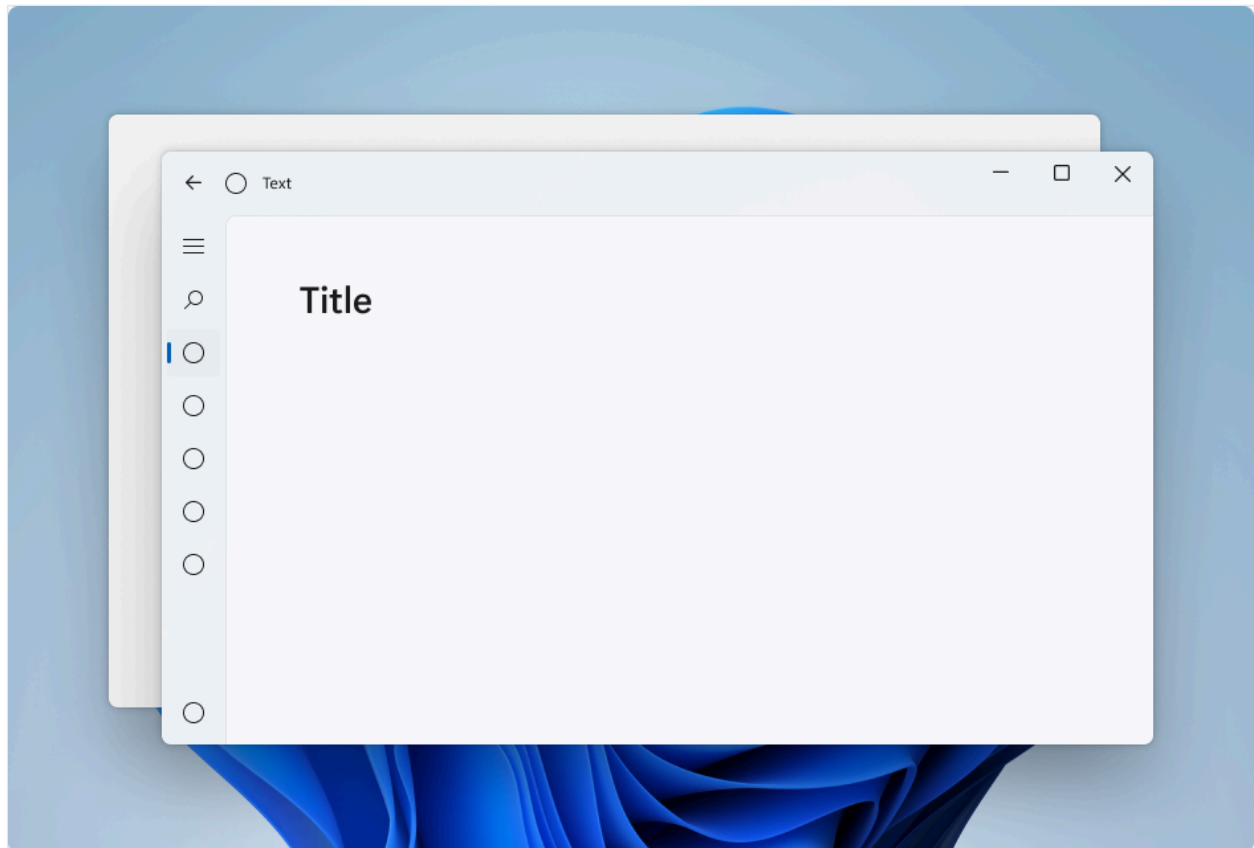
```
xmlns:muxc="using:Microsoft.UI.Xaml.Controls"

<Page muxc:BackdropMaterial.ApplyToRootOrPageBackground="True">
    <TextBlock>Hello world</TextBlock>
</Page>
```

The following examples show how to implement the standard layering patterns shown in the [Mica material](#) overview. Each of these examples use and require the same title bar code-behind, which is shown in the last example.

Example: Standard pattern in Left NavigationView

By default, [NavigationView](#) in Left mode includes the content layer in its content area. This example extends Mica into the title bar area and creates a custom title bar.



XAML

```
<Page
    x:Class="LeftNavView.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:LeftNavView"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:muxc="using:Microsoft.UI.Xaml.Controls"
    mc:Ignorable="d"
    muxc:BackdropMaterial.ApplyToRootOrPageBackground="True">
    <Page.Resources>
        <!--This top margin is the height of the custom TitleBar-->
        <Thickness x:Key="NavigationViewContentMargin">0,48,0,0</Thickness>
    </Page.Resources>
    <Grid>
        <Border x:Name="AppTitleBar"
            IsHitTestVisible="True"
```

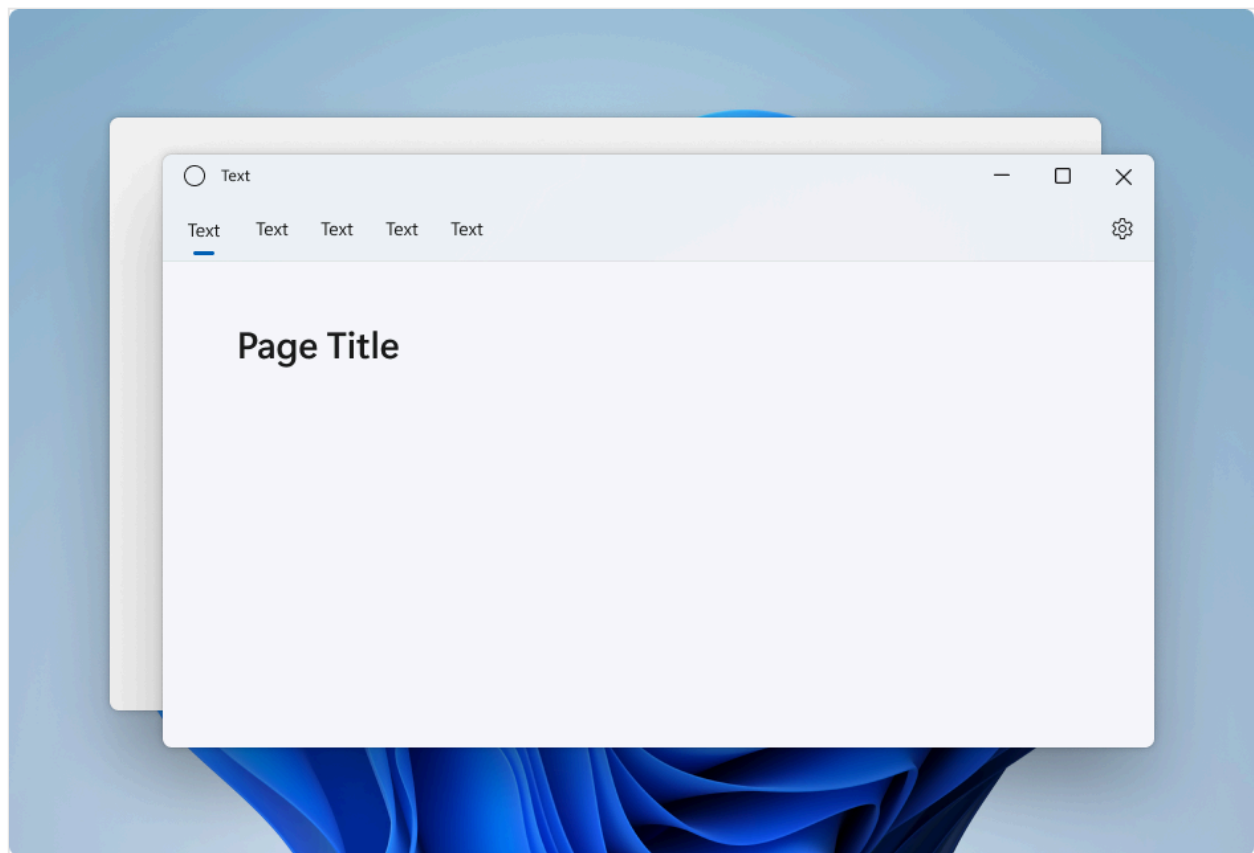
```

        VerticalAlignment="Top"
        Background="Transparent"
        Height="48"
        Canvas.ZIndex="1"
        Margin="48,0,0,0">
<StackPanel Orientation="Horizontal">
    <Image x:Name="AppFontIcon"
        HorizontalAlignment="Left"
        VerticalAlignment="Center"
        Source="Assets/Square44x44Logo.png"
        Width="16"
        Height="16"/>
    <TextBlock x:Name="AppTitle"
        Text="Test App Title"
        VerticalAlignment="Center"
        Margin="12, 0, 0, 0"
        Style="{StaticResource CaptionTextBlockStyle}" />
</StackPanel>
</Border>
<muxc:NavigationView x:Name="NavigationViewControl"
    IsTitleBarAutoPaddingEnabled="False"
    IsBackButtonVisible="Visible"
    Header="Title"
    DisplayModeChanged="NavigationViewControl_DisplayModeChanged"
    Canvas.ZIndex="0">
<muxc:NavigationView.MenuItems>
    <muxc:NavigationViewItem Icon="Target" Content="Text"/>
    <muxc:NavigationViewItem Icon="Target" Content="Text"/>
    <muxc:NavigationViewItem Icon="Target" Content="Text"/>
    <muxc:NavigationViewItem Icon="Target" Content="Text"/>
    <muxc:NavigationViewItem Icon="Target" Content="Text"/>
</muxc:NavigationView.MenuItems>
<Grid>
    <Frame x:Name="contentFrame">
        <Grid>
            <TextBlock Padding="56,16,0,0">Page content!
        </TextBlock>
        </Grid>
    </Frame>
</Grid>
</muxc:NavigationView>
</Grid>
</Page>

```

Example: Standard pattern in Top NavigationView

By default, [NavigationView](#) in Top mode includes the content layer in its content area. This example extends Mica into the title bar area and creates a custom title bar.



XAML

```
<Page
  x:Class="TopNavigationView.MainPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:TopNavigationView"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:muxc="using:Microsoft.UI.Xaml.Controls"
  mc:Ignorable="d"
  muxc:BackdropMaterial.ApplyToRootOrPageBackground="True">
  <Page.Resources>
    <CornerRadius
      x:Key="NavigationViewContentGridCornerRadius">0</CornerRadius>
  </Page.Resources>
  <Grid>
    <Border x:Name="AppTitleBar"
      IsHitTestVisible="True"
      VerticalAlignment="Top"
      Background="Transparent"
      Height="32"
      Margin="48,0,0,0">
      <StackPanel Orientation="Horizontal">
        <Image x:Name="AppFontIcon"
          HorizontalAlignment="Left"
          VerticalAlignment="Center"
          Source="Assets/Square44x44Logo.png"
          Width="16"
          Height="16"/>
        <TextBlock x:Name="AppTitle"
```



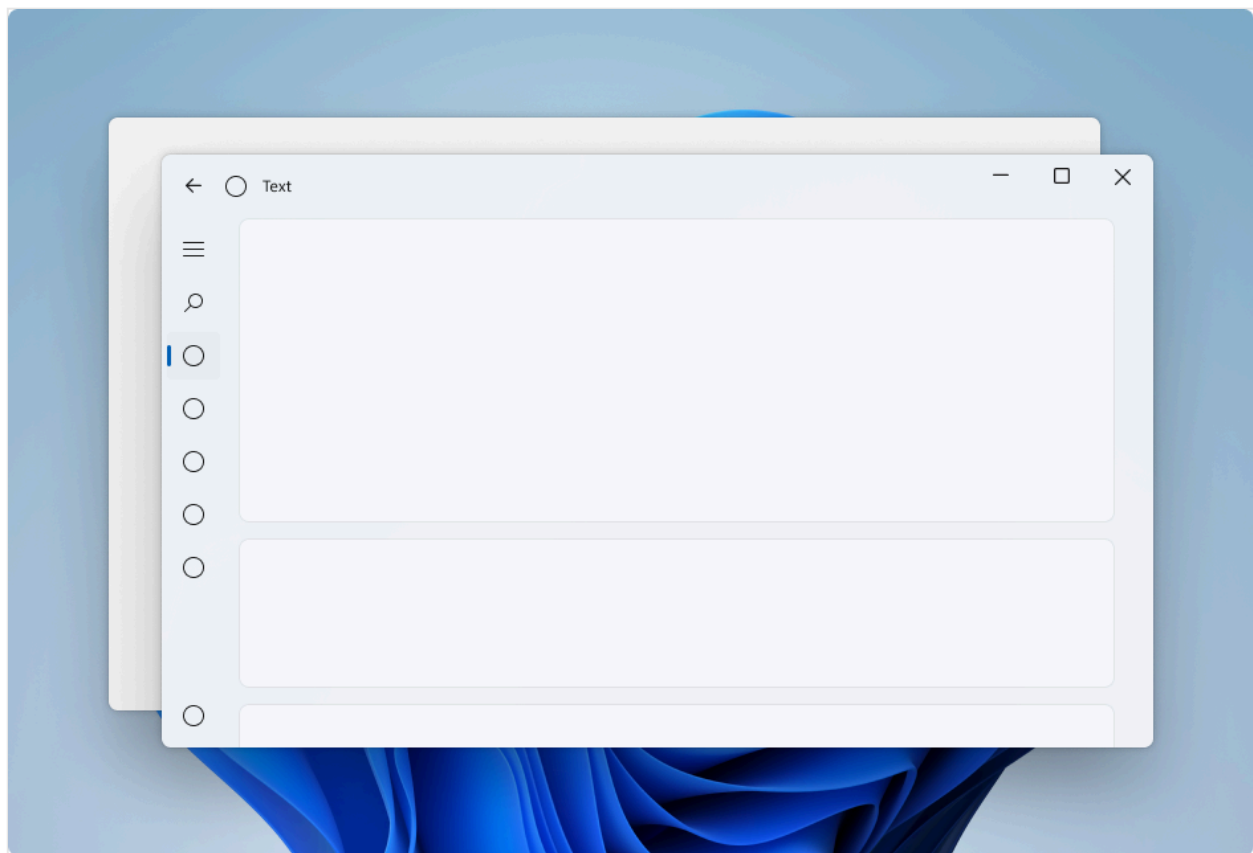
```

        Text="Test App Title"
        VerticalAlignment="Center"
        Margin="12,0,0,0"
        Style="{StaticResource CaptionTextBlockStyle}" />
    </StackPanel>
</Border>
<muxc:NavigationView x:Name="NavigationViewControl"
    Header="Page Title"
    DisplayModeChanged="NavigationViewControl_DisplayModeChanged"
    PaneDisplayMode="Top">
    <muxc:NavigationView.MenuItems>
        <muxc:NavigationViewItem Content="Text"/>
        <muxc:NavigationViewItem Content="Text"/>
        <muxc:NavigationViewItem Content="Text"/>
        <muxc:NavigationViewItem Content="Text"/>
        <muxc:NavigationViewItem Content="Text"/>
    </muxc:NavigationView.MenuItems>
    <Grid>
        <Frame x:Name="contentFrame">
            <Grid>
                <TextBlock Padding="56,16,0,0">Page content!
</TextBlock>
            </Grid>
        </Frame>
    </Grid>
</muxc:NavigationView>
</Grid>
</Page>

```

Example: Card pattern in Left NavigationView

To follow the card pattern using a [NavigationView](#) you will need to remove the default content layer by overriding the background and border theme resources. Then, you can create the cards in the content area of the control. This example creates several cards, extends Mica into the title bar area, and creates a custom title bar. For more information on card UI, see [Layering and Elevation](#) guidance.



XAML

<Page

```

x:Class="CardLayout.MainPage"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="using:CardLayout"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:muxc="using:Microsoft.UI.Xaml.Controls"
mc:Ignorable="d"
muxc:BackdropMaterial.ApplyToRootOrPageBackground="True">
    <Page.Resources>
        <!--This top margin is the height of the custom TitleBar-->
        <Thickness x:Key="NavigationViewContentMargin">0,48,0,0</Thickness>
        <Thickness
x:Key="NavigationViewContentGridBorderThickness">0</Thickness>
        <SolidColorBrush x:Key="NavigationViewContentBackground"
Color="Transparent"></SolidColorBrush>
    </Page.Resources>

```

<Grid>

```

    <Border x:Name="AppTitleBar"
        IsHitTestVisible="True"
        VerticalAlignment="Top"
        Background="Transparent"
        Height="48"
        Canvas.ZIndex="1"
        Margin="48,0,0,0">
        <StackPanel Orientation="Horizontal">
            <Image x:Name="AppFontIcon"

```

```

        HorizontalAlignment="Left"
        VerticalAlignment="Center"
        Source="Assets/Square44x44Logo.png"
        Width="16"
        Height="16"/>
<TextBlock x:Name="AppTitle"
    Text="Test App Title"
    VerticalAlignment="Center"
    Margin="12,0,0,0"
    Style="{StaticResource CaptionTextBlockStyle}" />
</StackPanel>
</Border>

<muxc:NavigationView x:Name="NavigationViewControl"
    IsTitleBarAutoPaddingEnabled="False"
    IsBackButtonVisible="Visible"
    Header="Title"
    DisplayModeChanged="NavigationViewControl_DisplayModeChanged"
    Canvas.ZIndex="0">
    <muxc:NavigationView.MenuItems>
        <muxc:NavigationViewItem Icon="Target" Content="Text"/>
        <muxc:NavigationViewItem Icon="Target" Content="Text"/>
        <muxc:NavigationViewItem Icon="Target" Content="Text"/>
        <muxc:NavigationViewItem Icon="Target" Content="Text"/>
        <muxc:NavigationViewItem Icon="Target" Content="Text"/>
    </muxc:NavigationView.MenuItems>
    <Grid>
        <Frame x:Name="contentFrame">
            <StackPanel Orientation="Vertical" Margin="40,16,0,0">
                <Border Width="600" Height="200" Background="
{ThemeResource LayerFillColorDefaultBrush}"
                    VerticalAlignment="Top"
                    HorizontalAlignment="Left"
                    Margin="16"
                    CornerRadius="8"
                    BorderThickness="1"
                    BorderBrush="{ThemeResource
CardStrokeColorDefaultBrush}">
                    <TextBlock VerticalAlignment="Center"
HorizontalAlignment="Center">Content here!</TextBlock>
                </Border>
                <Border Width="600" Height="200" Background="
{ThemeResource LayerFillColorDefaultBrush}"
                    VerticalAlignment="Top"
                    HorizontalAlignment="Left"
                    Margin="16"
                    CornerRadius="8"
                    BorderThickness="1"
                    BorderBrush="{ThemeResource
CardStrokeColorDefaultBrush}">
                    <TextBlock VerticalAlignment="Center"
HorizontalAlignment="Center">Content here!</TextBlock>
                </Border>
                <Border Width="600" Height="200" Background="
{ThemeResource LayerFillColorDefaultBrush}"

```

```

        VerticalAlignment="Top"
        HorizontalAlignment="Left"
        Margin="16"
        CornerRadius="8"
        BorderThickness="1"
        BorderBrush="{ThemeResource
CardStrokeColorDefaultBrush}">
        <TextBlock VerticalAlignment="Center"
HorizontalAlignment="Center">Content here!</TextBlock>
    </Border>
</StackPanel>
</Frame>
</Grid>
</muxc:NavigationView>
</Grid>
</Page>

```

Title bar code-behind

The previous three app layout XAML pages use this code-behind to create a custom title bar adaptive to app state and visibility.

For more information see [Title bar customization](#).

C#

```

public MainPage()
{
    this.InitializeComponent();
    var titleBar = ApplicationView.GetForCurrentView().TitleBar;

    titleBar.ButtonBackgroundColor = Colors.Transparent;
    titleBar.ButtonInactiveBackgroundColor = Colors.Transparent;

    // Hide default title bar.
    var coreTitleBar = CoreApplication.GetCurrentView().TitleBar;
    coreTitleBar.ExtendViewIntoTitleBar = true;
    UpdateTitleBarLayout(coreTitleBar);

    // Set XAML element as a draggable region.
    Window.Current.SetTitleBar(AppTitleBar);

    // Register a handler for when the size of the overlaid caption control
    changes.
    // For example, when the app moves to a screen with a different DPI.
    coreTitleBar.LayoutMetricsChanged += CoreTitleBar_LayoutMetricsChanged;

    // Register a handler for when the title bar visibility changes.
    // For example, when the title bar is invoked in full screen mode.
    coreTitleBar.IsVisibleChanged += CoreTitleBar_IsVisibleChanged;
}

```

```

        //Register a handler for when the window changes focus
        Window.Current.Activated += Current_Activated;
    }

    private void CoreTitleBar_LayoutMetricsChanged(CoreApplicationViewTitleBar
sender, object args)
    {
        UpdateTitleBarLayout(sender);
    }

    private void UpdateTitleBarLayout(CoreApplicationViewTitleBar coreTitleBar)
    {
        // Update title bar control size as needed to account for system size
        changes.
        AppTitleBar.Height = coreTitleBar.Height;

        // Ensure the custom title bar does not overlap window caption controls
        Thickness currMargin = AppTitleBar.Margin;
        AppTitleBar.Margin = new Thickness(currMargin.Left, currMargin.Top,
coreTitleBar.SystemOverlayRightInset, currMargin.Bottom);
    }

    private void CoreTitleBar_IsVisibleChanged(CoreApplicationViewTitleBar
sender, object args)
    {
        if (sender.IsVisible)
        {
            AppTitleBar.Visibility = Visibility.Visible;
        }
        else
        {
            AppTitleBar.Visibility = Visibility.Collapsed;
        }
    }

    // Update the TitleBar based on the inactive/active state of the app
    private void Current_Activated(object sender,
Windows.UI.Core.WindowActivatedEventArgs e)
    {
        SolidColorBrush defaultForegroundBrush =
(SolidColorBrush)Application.Current.Resources["TextFillColorPrimaryBrush"];
        SolidColorBrush inactiveForegroundBrush =
(SolidColorBrush)Application.Current.Resources["TextFillColorDisabledBrush"]
;

        if (e.WindowActivationState ==
Windows.UI.Core.CoreWindowActivationState.Deactivated)
        {
            AppTitle.Foreground = inactiveForegroundBrush;
        }
        else
        {
            AppTitle.Foreground = defaultForegroundBrush;
        }
    }
}

```

```

// Update the TitleBar content layout depending on NavigationView
DisplayMode
private void
NavigationViewControl_DisplayModeChanged(Microsoft.UI.Xaml.Controls.Navigati
onView sender,
Microsoft.UI.Xaml.Controls.NavigationViewDisplayModeChangedEventArgs args)
{
    const int topIndent = 16;
    const int expandedIndent = 48;
    int minimalIndent = 104;

    // If the back button is not visible, reduce the TitleBar content
    indent.
    if
    (NavigationViewControl.IsBackButtonVisible.Equals(Microsoft.UI.Xaml.Controls
.NavigationViewBackButtonVisible.Collapsed))
    {
        minimalIndent = 48;
    }

    Thickness currMargin = AppTitleBar.Margin;

    // Set the TitleBar margin dependent on NavigationView display mode
    if (sender.PaneDisplayMode ==
Microsoft.UI.Xaml.Controls.NavigationViewPaneDisplayMode.Top)
    {
        AppTitleBar.Margin = new Thickness(topIndent, currMargin.Top,
currMargin.Right, currMargin.Bottom);
    }
    else if (sender.DisplayMode ==
Microsoft.UI.Xaml.Controls.NavigationViewDisplayMode.Minimal)
    {
        AppTitleBar.Margin = new Thickness(minimalIndent, currMargin.Top,
currMargin.Right, currMargin.Bottom);
    }
    else
    {
        AppTitleBar.Margin = new Thickness(expandedIndent, currMargin.Top,
currMargin.Right, currMargin.Bottom);
    }
}

```

Related articles

- [Mica material](#)
- [Materials](#)
- [Layering and Elevation](#)
- [Apply Mica or Acrylic materials in desktop apps for Windows 11](#)
- [Apply Mica in Win32 desktop apps for Windows 11](#)

- [BackdropMaterial class](#)
- [NavigationView](#)

Feedback

Was this page helpful?



Yes

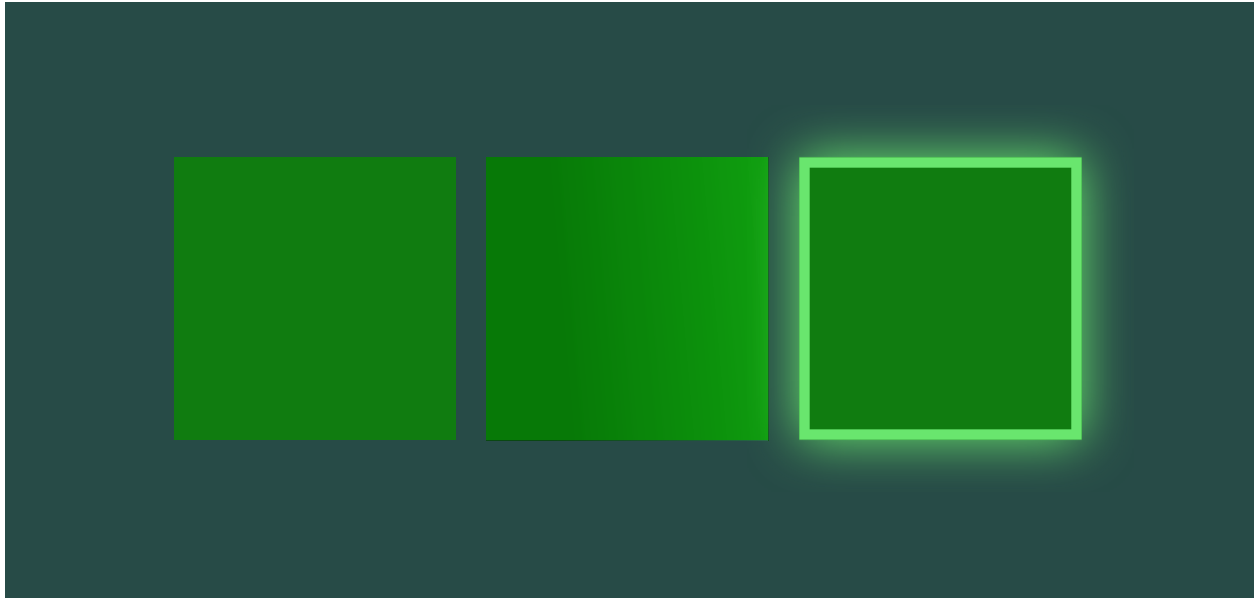


No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Reveal Focus

Article • 11/26/2024



Reveal Focus is a lighting effect for 10-foot experiences, such as game consoles on television screens. It animates the border of focusable elements, such as buttons, when the user moves gamepad or keyboard focus to them. It's turned off by default, but it's simple to enable.

Important APIs: [Application.FocusVisualKind property](#), [FocusVisualKind enum](#), [Control.UseSystemFocusVisuals property](#)

How it works

Reveal Focus calls attention to focused elements by adding an animated glow around the element's border:



This is especially helpful in 10-foot scenarios where the user might not be paying full attention to the entire TV screen.

Examples

✓ [Open the WinUI 2 Gallery app and see RevealFocus in action.](#)

The **WinUI 2 Gallery** app includes interactive examples of most WinUI 2 controls, features, and functionality. Get the app from the [Microsoft Store](#) or get the source code on [GitHub](#).

How to use it

Reveal Focus is off by default. To enable it:

1. In your app's constructor, call the [AnalyticsInfo.VersionInfo.DeviceFamily](#) property and check whether the current device family is `Windows.Xbox`.
2. If the device family is `Windows.Xbox`, set the [Application.FocusVisualKind](#) property to `FocusVisualKind.Reveal`.

C#

```
if (AnalyticsInfo.VersionInfo.DeviceFamily == "Windows.Xbox")
{
    this.FocusVisualKind = FocusVisualKind.Reveal;
}
```

After you set the `FocusVisualKind` property, the system automatically applies the Reveal Focus effect to all controls whose `UseSystemFocusVisuals` property is set to `True` (the default value for most controls).

Why isn't Reveal Focus on by default?

As you can see, it's fairly easy to turn on Reveal Focus when the app detects it's running on an Xbox. So, why doesn't the system just turn it on for you? Because Reveal Focus increases the size of the focus visual, which might cause issues with your UI layout. In some cases, you'll want to customize the Reveal Focus effect to optimize it for your app.

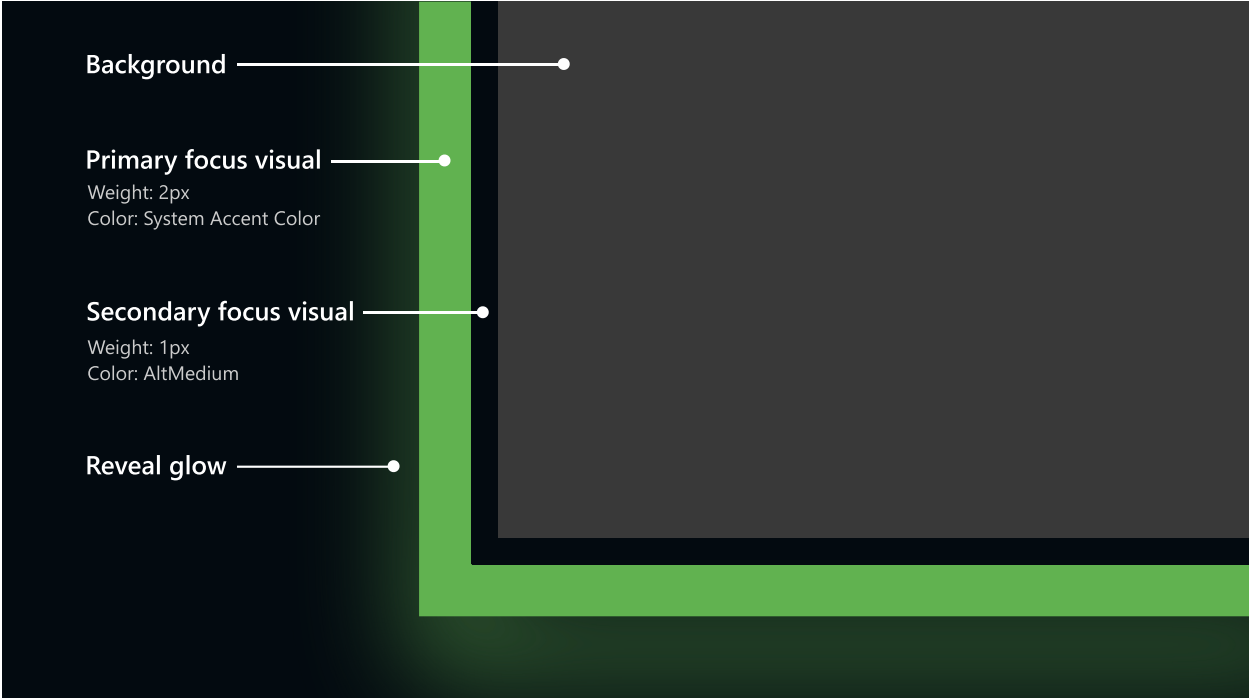
Customizing Reveal Focus

You can customize the Reveal Focus effect by modifying the focus visual properties for each control: `FocusVisualPrimaryThickness`, `FocusVisualSecondaryThickness`, `FocusVisualPrimaryBrush`, and `FocusVisualSecondaryBrush`. These properties let you customize the color and thickness of the focus rectangle. (They're the same properties you use for creating [High Visibility focus visuals](#).)

But before you start customizing it, it's helpful to know a little more about the components that make up Reveal Focus.

There are three parts to the default Reveal Focus visuals: the primary border, the secondary border and the Reveal glow. The primary border is **2px** thick, and runs around the *outside* of the secondary border. The secondary border is **1px** thick and runs around the *inside* of the primary border. The Reveal Focus glow has a thickness proportional to the thickness of the primary border and runs around the *outside* of the primary border.

In addition to the static elements, Reveal Focus visuals feature an animated light that pulsates when at rests and moves in the direction of focus when moving focus.



Customize the border thickness

To change the thickness of the border types of a control, use these properties:

[Expand table](#)

Border type	Property
Primary, Glow	FocusVisualPrimaryThickness (Changing the primary border changes the thickness of the glow proportionately.)
Secondary	FocusVisualSecondaryThickness

This example changes the border thickness of a button's focus visual:

XAML

```
<Button FocusVisualPrimaryThickness="2" FocusVisualSecondaryThickness="1"/>
```

Customize the margin

The margin is the space between the control's visual bounds and the start of the focus visuals secondary border. The default margin is 1px away from the control bounds. You can edit this margin on a per-control basis by changing the [FocusVisualMargin](#) property:

XAML

```
<Button FocusVisualPrimaryThickness="2" FocusVisualSecondaryThickness="1"
FocusVisualMargin="-3"/>
```

A negative margin pushes the border away from the center of the control, and a positive margin moves the border closer to the center of the control.

Customize the color

To change color of the Reveal Focus visual, use the `FocusVisualPrimaryBrush` and `FocusVisualSecondaryBrush` properties.

 Expand table

Property	Default resource	Default resource value
<code>FocusVisualPrimaryBrush</code>	<code>SystemControlRevealFocusVisualBrush</code>	<code>SystemAccentColor</code>
<code>FocusVisualSecondaryBrush</code>	<code>SystemControlFocusVisualSecondaryBrush</code>	<code>SystemAltMediumColor</code>

(The `FocusPrimaryBrush` property only defaults to the `SystemControlRevealFocusVisualBrush` resources when `FocusVisualKind` is set to `Reveal`. Otherwise, it uses `SystemControlFocusVisualPrimaryBrush`.)

To change the color of the focus visual of an individual control, set the properties directly on the control. This example overrides the focus visual colors of a button.

XAML

```
<!-- Specifying a color directly -->
<Button
    FocusVisualPrimaryBrush="DarkRed"
    FocusVisualSecondaryBrush="Pink"/>

<!-- Using theme resources -->
<Button
    FocusVisualPrimaryBrush="{ThemeResource SystemBaseHighColor}"
    FocusVisualSecondaryBrush="{ThemeResource SystemAccentColor}"/>
```

To change the color of all focus visuals in your entire app, override the `SystemControlRevealFocusVisualBrush` and `SystemControlFocusVisualSecondaryBrush` resources with your own definition:

XAML

```

<!-- App.xaml -->
<Application.Resources>

    <!-- Override Reveal Focus default resources. -->
    <SolidColorBrush x:Key="SystemControlRevealFocusVisualBrush" Color="
{ThemeResource SystemBaseHighColor}"/>
    <SolidColorBrush x:Key="SystemControlFocusVisualSecondaryBrush" Color="
{ThemeResource SystemAccentColor}"/>
</Application.Resources>

```

For more information on modifying the focus visual's color, see [Color Branding & Customizing](#).

Show just the glow

If you'd like to use only the glow without the primary or secondary focus visual, simply set the control's [FocusVisualPrimaryBrush](#) property to `Transparent` and the [FocusVisualSecondaryThickness](#) to `0`. In this case, the glow will adopt the color of the control background to provide a borderless feel. You can modify the thickness of the glow using [FocusVisualPrimaryThickness](#).

XAML

```

<!-- Show just the glow -->
<Button
    FocusVisualPrimaryBrush="Transparent"
    FocusVisualSecondaryThickness="0" />

```

Use your own focus visuals

Another way to customize Reveal Focus is to opt out of the system-provided focus visuals by drawing your own using visual states. To learn more, see the [Focus visuals sample](#).

Related articles

- [Gamepad and remote control interactions](#)
- [Focus visuals sample](#)
- [Composition Effects](#)
- [Science in the System: Fluent Design and Depth](#)
- [Science in the System: Fluent Design and Light](#)

Feedback

Was this page helpful?



Yes

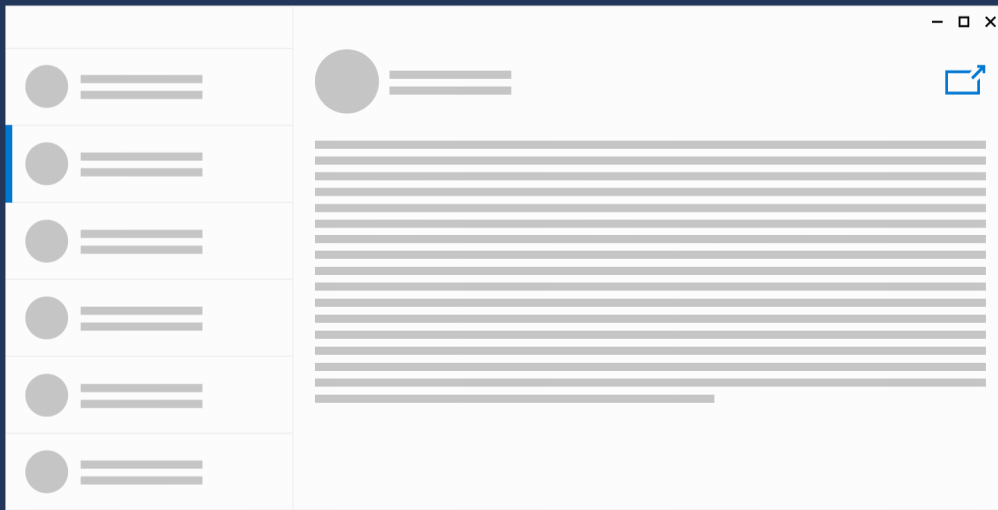


No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Show multiple views for an app

Article • 11/26/2024



Help your users be more productive by letting them view independent parts of your app in separate windows. When you create multiple windows for an app, the taskbar shows each window separately. Users can move, resize, show, and hide app windows independently and can switch between app windows as if they were separate apps.

Important APIs: [Windows.UI.ViewManagement namespace](#), [Windows.UI.WindowManagement namespace](#)

When should an app use multiple views?

There are a variety of scenarios that can benefit from multiple views. Here are a few examples:

- An email app that lets users view a list of received messages while composing a new email
- An address book app that lets users compare contact info for multiple people side-by-side
- A music player app that lets users see what's playing while browsing through a list of other available music
- A note-taking app that lets users copy information from one page of notes to another
- A reading app that lets users open several articles for reading later, after an opportunity to peruse all high-level headlines

While each app layout is unique, we recommend including a "new window" button in a predictable location, such as the top right corner of the content that can be opened in a new window. Also consider including a [context menu](#) option to "Open in a new window".

To create separate instances of your app (rather than separate windows for the same instance), see [Create a multi-instance Windows app](#).

Windowing hosts

There are different ways that Windows content can be hosted inside an app.

- [CoreApplicationView](#)

An app view is the 1:1 pairing of a thread and a window that the app uses to display content. The first view that's created when your app starts is called the *main view*. Each CoreWindow/ApplicationWindow operates in its own thread. Having to work on different UI threads can complicate multi-window apps.

The main view for your app is always hosted in an ApplicationWindow. Content in a secondary window can be hosted in a ApplicationWindow or in an AppWindow.

To learn how to use ApplicationWindow to show secondary windows in your app, see [Use ApplicationWindow](#).

- [AppWindow](#)

AppWindow simplifies the creation of multi-window Windows apps because it operates on the same UI thread that it's created from.

The AppWindow class and other APIs in the [WindowManagement](#) namespace are available starting in Windows 10, version 1903 (SDK 18362). If your app targets earlier versions of Windows 10, you must use ApplicationWindow to create secondary windows.

To learn how to use AppWindow to show secondary windows in your app, see [Use AppWindow](#).

⚠ **Note**

Some platform and framework components are known to not work with AppWindow (see [Limitations](#)).

- [DesktopWindowXamlSource](#) (XAML Islands)

UWP XAML content in a Win32 app (using HWND), also known as XAML Islands, is hosted in a DesktopWindowXamlSource.

For more info about XAML Islands, see [Using the UWP XAML hosting API in a desktop application](#)

Make code portable across windowing hosts

When XAML content is displayed in a [CoreApplicationView](#), there's always an associated [ApplicationWindow](#) and XAML [Window](#). You can use APIs on these classes to get information such as the window bounds. To retrieve an instance of these classes, you use the static [CoreApplicationView.GetForCurrentThread](#) method, [ApplicationWindow.GetForCurrentView](#) method, or [Window.Current](#) property. In addition, there are many classes that use the `GetForCurrentView` pattern to retrieve an instance of the class, such as [DisplayInformation.GetForCurrentView](#).

These APIs work because there is only a single tree of XAML content for a CoreWindow/ApplicationWindow, so the XAML knows the context in which it's hosted is that CoreWindow/ApplicationWindow.

When XAML content is running inside an `AppWindow` or `DesktopWindowXamlSource`, you can have multiple trees of XAML content running on the same thread at the same time. In this case, these APIs don't give the right information, since the content is no longer running inside the current `CoreWindow/ApplicationView` (and XAML Window).

To ensure that your code works correctly across all windowing hosts, you should replace APIs that rely on `CoreWindow`, `ApplicationView`, and `Window` with new APIs that get their context from the `XamlRoot` class. The `XamlRoot` class represents a tree of XAML content and information about the context in which it is hosted, whether it's a `CoreWindow`, `AppWindow`, or `DesktopWindowXamlSource`. This abstraction layer lets you write the same code regardless of which windowing host the XAML runs in.

This table shows code that does not work correctly across windowing hosts, and the new portable code that you can replace it with, as well as some APIs that you don't need to change.

 Expand table

If you used...	Replace with...
<code>CoreWindow.GetForCurrentThread().Bounds</code>	<code>uiElement.XamlRoot.Size</code>
<code>CoreWindow.GetForCurrentThread().SizeChanged</code>	<code>uiElement.XamlRoot.Changed</code>
<code>CoreWindow.Visible</code>	<code>uiElement.XamlRoot.IsHostVisible</code>
<code>CoreWindow.VisibilityChanged</code>	<code>uiElement.XamlRoot.Changed</code>
<code>CoreWindow.GetForCurrentThread().GetKeyState</code>	Unchanged. This is supported in <code>AppWindow</code> and <code>DesktopWindowXamlSource</code> .
<code>CoreWindow.GetForCurrentThread().GetAsyncKeyState</code>	Unchanged. This is supported in <code>AppWindow</code> and <code>DesktopWindowXamlSource</code> .
<code>Window.Current</code>	Returns the main XAML Window object which is closely bound to the current <code>CoreWindow</code> . See Note after this table.
<code>Window.Current.Bounds</code>	<code>uiElement.XamlRoot.Size</code>
<code>Window.Current.Content</code>	UIElement root = <code>uiElement.XamlRoot.Content</code>
<code>Window.Current.Compositor</code>	Unchanged. This is supported in <code>AppWindow</code> and <code>DesktopWindowXamlSource</code> .
<code>VisualTreeHelper.FindElementsInHostCoordinates</code> Although the <code>UIElement</code> param is optional, the method raises an exception if a <code>UIElement</code> isn't supplied when hosted on an Island.	Specify the <code>uiElement.XamlRoot</code> as <code>UIElement</code> instead of leaving it blank.
<code>VisualTreeHelper.GetOpenPopups</code> In XAML Islands apps this will throw an error. In <code>AppWindow</code> apps this will return open popups on the main window.	<code>VisualTreeHelper.GetOpenPopupsForXamlRoot(uiElement.XamlRoot)</code>
<code>FocusManager.GetFocusedElement</code>	<code>FocusManager.GetFocusedElement(uiElement.XamlRoot)</code>
<code>contentDialog.ShowAsync()</code>	<code>contentDialog.XamlRoot = uiElement.XamlRoot;</code> <code>contentDialog.ShowAsync();</code>

If you used...	Replace with...
<code>menuFlyout.ShowAt(null, new Point(10, 10));</code>	<pre>menuFlyout.XamlRoot = uiElement.XamlRoot; menuFlyout.ShowAt(null, new Point(10, 10));</pre>

ⓘ Note

For XAML content in a `DesktopWindowXamlSource`, there does exist a `CoreWindow/Window` on the thread, but it is always invisible and has a size of 1x1. It is still accessible to the app but won't return meaningful bounds or visibility.

For XAML content in an `AppWindow`, there will always be exactly one `CoreWindow` on the same thread. If you call a `GetForCurrentView` or `GetForCurrentThread` API, that API will return an object that reflects the state of the `CoreWindow` on the thread, not any of the `AppWindows` that may be running on that thread.

Do's and don'ts

- Do provide a clear entry point to the secondary view by utilizing the "open new window" glyph.
- Do communicate the purpose of the secondary view to users.
- Do ensure that your app is fully functional in a single view and users will open a secondary view only for convenience.
- Don't rely on the secondary view to provide notifications or other transient visuals.

Related topics

- [Use AppWindow](#)
- [Use ApplicationView](#)
- [ApplicationViewSwitcher](#)
- [CreateNewView](#)

Feedback

Was this page helpful?

[Provide product feedback](#) [↗](#) | [Get help at Microsoft Q&A](#)

Show multiple views with AppWindow

Article • 11/26/2024

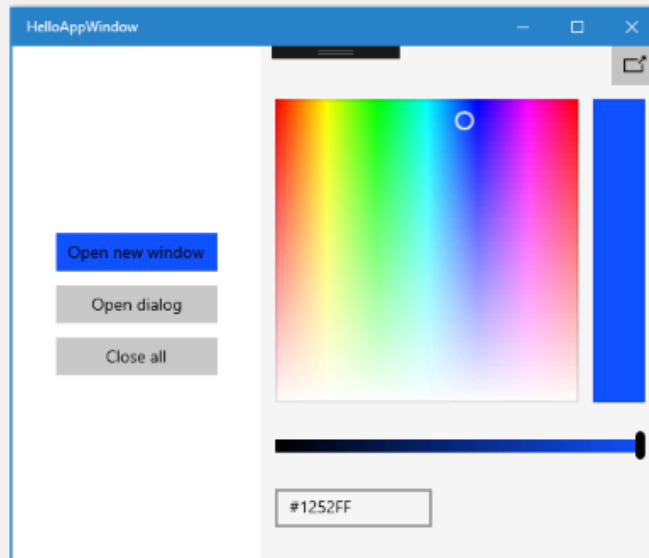
[AppWindow](#) and its related APIs simplify the creation of multi-window apps by letting you show your app content in secondary windows while still working on the same UI thread across each window.

ⓘ Note

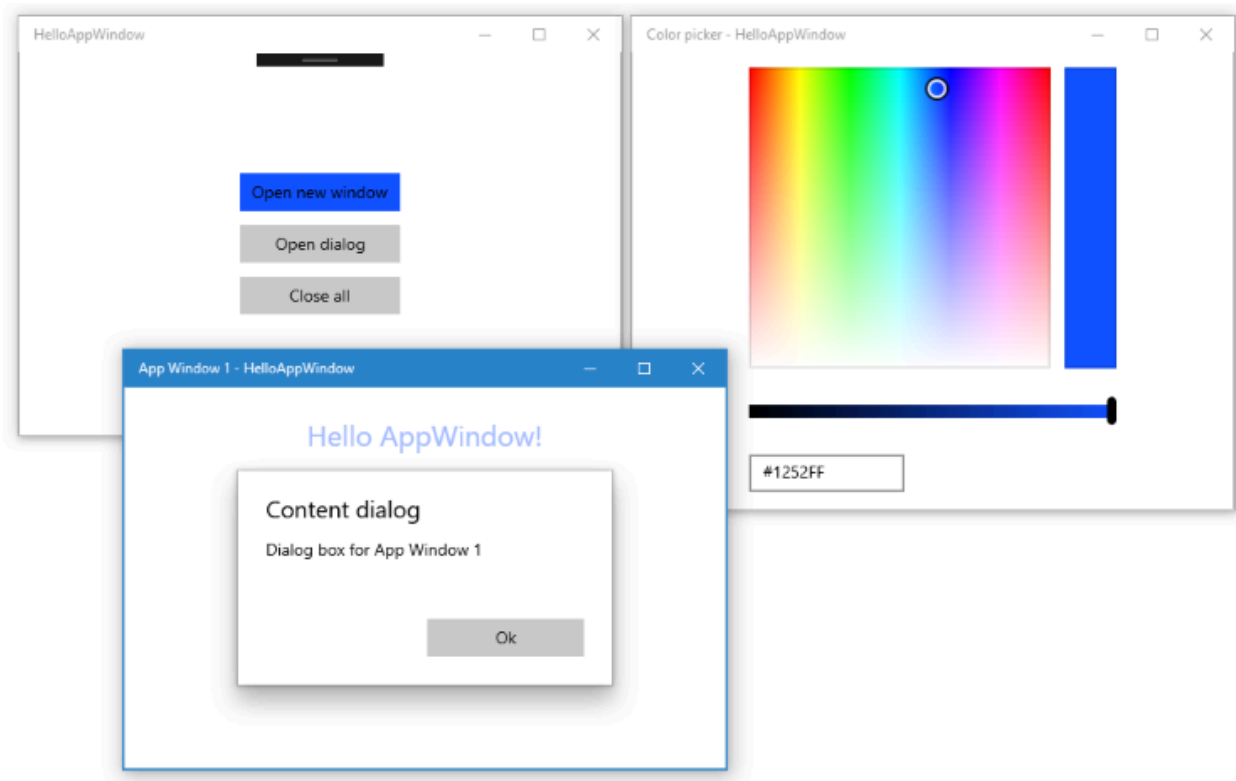
AppWindow is currently in preview. This means you can submit apps that use AppWindow to the Store, but some platform and framework components are known to not work with AppWindow (see [Limitations](#)).

Here, we show some scenarios for multiple windows with a sample app called `HelloAppWindow`. The sample app demonstrates the following functionality:

- Un-dock a control from the main page and open it in a new window.
- Open new instances of a Page in new windows.
- Programmatically size and position new windows in the app.
- Associate a ContentDialog with the appropriate window in the app.



Sample app with a single window



Sample app with un-docked color picker and secondary window

Important APIs: [Windows.UI.WindowManagement](#) namespace, [AppWindow](#) class

API overview

The [AppWindow](#) class and other APIs in the [WindowManagement](#) namespace are available starting in Windows 10, version 1903 (SDK 18362). If your app targets earlier versions of Windows 10, you must [use ApplicationView to create secondary windows](#). WindowManagement APIs are still under development and have [limitations](#) as described in the API reference docs.

Here are some of the important APIs you use to show content in an AppWindow.

AppWindow

The [AppWindow](#) class can be used to display a portion of a UWP app in a secondary window. It is similar in concept to an [ApplicationView](#), but not the same in behavior and lifetime. A main feature of AppWindow is that each instance shares the same UI processing thread (including the event dispatcher) from which they were created, which simplifies multi-window apps.

You can only connect XAML content to your AppWindow, there is no support for native DirectX or Holographic content. However, you can show a XAML [SwapChainPanel](#) that

hosts DirectX content.

WindowingEnvironment

The [WindowingEnvironment](#) API lets you know about the environment where your app is being presented so you can adapt your app as needed. It describes the kind of window the environment supports; for example, `Overlapped` if the app is running on a PC, or `Tiled` if the app is running on an Xbox. It also provides a set of `DisplayRegion` objects that describe the areas in which an app may be shown on a logical display.

DisplayRegion

The [DisplayRegion](#) API describes the region in which a view can be shown to a user on a logical display; for example, on a desktop PC, this is the full display minus the area of the taskbar. It is not necessarily a 1:1 mapping with the physical display area of the backing monitor. There can be multiple display regions within the same monitor, or a `DisplayRegion` can be configured to span across multiple monitors if those monitors are homogenous in all aspects.

AppWindowPresenter

The [AppWindowPresenter](#) API lets you easily switch windows into pre-defined configurations like `Fullscreen` or `CompactOverlay`. These configurations give the user a consistent experience across any device that supports the configuration.

UIContext

[UIContext](#) is a unique identifier for an app window or view. It's created automatically, and you can use the [UIElement.UIContext](#) property to retrieve the `UIContext`. Every `UIElement` in the XAML tree has the same `UIContext`.

`UIContext` is important because APIs like [Window.Current](#) and the `GetForCurrentView` pattern rely on having a single `ApplicationView/CoreWindow` with a single XAML tree per thread to work with. This isn't the case when you use an `AppWindow`, so you use `UIContext` to identify a particular window instead.

XamlRoot

The [XamlRoot](#) class holds a XAML element tree, connects it to the window host object (for example, the [AppWindow](#) or [ApplicationView](#)), and provides info such as size and